

Guide Docker : dockerisation générique

Dockeriser n'importe quel projet

Universel · Fonctionne avec n'importe quel langage ou framework

Prérequis : Docker Desktop installé et lancé

Lexique — Les concepts clés

Terme	Définition
Image	La "recette" d'un environnement. Immuable, téléchargée depuis hub.docker.com ou construite via un Dockerfile
Conteneur	Une instance en cours d'exécution d'une image. C'est la "boîte" qui fait tourner ton app
Dockerfile	Fichier d'instructions pour construire une image personnalisée
compose.yaml	Fichier qui orchestre plusieurs conteneurs ensemble
Volume	Stockage persistant. Les données survivent à l'arrêt du conteneur
Network	Réseau interne entre conteneurs. Ils se parlent via leur nom de service , jamais via <code>localhost</code>
Port mapping	<code>"8080:80"</code> → port sur ta machine : port dans le conteneur
<code>.env</code>	Fichier de variables d'environnement sensibles. Ne jamais committer sur Git
<code>.dockerignore</code>	Liste les fichiers à ne PAS copier dans l'image (comme <code>.gitignore</code>)
Cache Docker	Docker met en cache chaque étape du Dockerfile. Si rien n'a changé, il réutilise le cache → builds rapides
<code>depends_on</code>	Définit l'ordre de démarrage des services dans le compose
<code>build:</code>	Indique à Docker de construire l'image depuis un Dockerfile local
<code>image:</code>	Indique à Docker d'utiliser une image toute faite depuis Docker Hub

Structure cible universelle

```

mon-projet/
├── service-1/                ← ex: backend, api, worker...
│   ├── Dockerfile
│   └── .dockerignore
├── service-2/                ← ex: frontend, client...
│   ├── Dockerfile
│   └── .dockerignore
├── compose.yaml              ← orchestre tous les services
├── .env                      ← variables sensibles ⚠ ne jama
└── is committer !
    └── .gitignore

```

💡 Chaque service qui nécessite du code custom a son propre `Dockerfile`.

Les services tiers (base de données, cache...) utilisent directement une `image:` dans le compose.

Fichier par fichier

`.gitignore`

À créer **en premier**, avant tout commit.

```

# Variables d'environnement – CRITIQUE
.env
.env.*

# Dépendances (souvent très lourdes)
node_modules/
vendor/
__pycache__/
*.pyc

# Builds générés
dist/
build/

```

```
.next/  
out/
```



.env

Contient toutes les **variables sensibles**. Jamais sur Git.

```
# — <SERVICE_1> —————  
<VARIABLE_1>=<valeur>  
<VARIABLE_2>=<valeur>  
  
# — Base de données —————  
DB_HOST=<nom_du_service_db_dans_le_compose>  
DB_PORT=<port_par_defaut_de_ta_db>  
DB_USER=<utilisateur>  
DB_PASSWORD=<mot_de_passe>  
DB_NAME=<nom_de_la_base>
```

⚠ Règles strictes :

- Pas d'espaces autour du `=`
- Pas de guillemets (sauf si la valeur contient des espaces)
- `DB_HOST` = le **nom du service** dans le compose, jamais `localhost`



Dockerfile — Structure universelle

```
# — ÉTAPE 1 : Image de base —————  
_____  
FROM <image>:<version>  
# Choisir sur https://hub.docker.com  
# Toujours préciser une version. Éviter "latest" (non repro  
ductible)  
# Variante "alpine" = image légère (~10x plus légère) mais  
peut manquer d'outils natifs  
# Variante standard (ubuntu/debian) = plus lourde mais zéro  
prise de tête
```

```

# — ÉTAPE 2 : Répertoire de travail —————
WORKDIR /app
# Crée le dossier /app et s'y positionne.
# Toutes les commandes suivantes s'exécutent depuis /app.
# Convention : /app pour les apps web, /usr/src/app aussi c
ourant.

# — ÉTAPE 3 : Dépendances (AVANT le code source) —————
COPY <fichier_dependances> ./
# ex Node.js : COPY package*.json ./
# ex Python : COPY requirements.txt ./
# ex PHP : COPY composer.json composer.lock ./
# ex Ruby : COPY Gemfile Gemfile.lock ./

RUN <commande_installation>
# ex Node.js : RUN npm install
# ex Python : RUN pip install -r requirements.txt
# ex PHP : RUN composer install
# ex Ruby : RUN bundle install
#
# ✂ ASTUCE CACHE : en copiant les fichiers de dépendances E
N PREMIER,
# Docker met en cache cette étape. Si ton code change mais
pas tes dépendances,
# l'installation est réutilisée → build 10x plus rapide !

# — ÉTAPE 4 : Code source —————
COPY . .
# Copie tout le reste du dossier dans /app
# Le .dockerignore filtre ce qui ne doit pas être copié

# — ÉTAPE 5 : Port exposé —————
EXPOSE <port>
# Documente le port d'écoute de l'application.

```

```
# Informatif uniquement – c'est le compose.yaml qui ouvre v  
raiment le port.
```

```
# — ÉTAPE 6 : Commande de démarrage —————
```

```
CMD ["<commande>", "<argument1>", "<argument2>"]  
# Chaque argument est séparé dans le tableau.  
# ex Node.js dev : CMD ["npm", "run", "dev"]  
# ex Python      : CMD ["python", "app.py"]  
# ex Go          : CMD ["/mon-binaire"]
```

.dockerignore

À placer dans le **même dossier** que chaque Dockerfile.

```
# Dépendances – CRITIQUE (peut peser des centaines de Mo)  
node_modules  
vendor  
__pycache__  
*.pyc  
  
# Builds générés (Docker les reconstruira)  
dist  
build  
.next  
out  
  
# Variables sensibles  
.env  
.env.*  
  
# Fichiers inutiles  
.git  
.gitignore  
*.md  
.DS_Store  
Thumbs.db
```

```
# Logs
*.log
```

⚠ Sans `.dockerignore`, le `COPY . .` copie **tout** dans l'image, y compris `node_modules` (500 Mo) ou ton `.env` avec tes mots de passe. C'est l'erreur la plus courante chez les débutants.

`compose.yaml` — Structure universelle

```
services:

  # — Service avec Dockerfile custom —————
  <nom_service_1>:
    build: ./<dossier_du_service>
    # Pointe vers le dossier contenant le Dockerfile
    # Docker cherche automatiquement "Dockerfile" dedans

    ports:
      - "<port_machine>:<port_conteneur>"
      # Ce qui est à gauche du ":" = port sur TON PC (chois
      is librement)
      # Ce qui est à droite = port dans le conteneur (défini
      i par l'app)

    depends_on:
      - <nom_service_dont_il_depends>
      # Démarre ce service APRÈS les services listés ici

    environment:
      <VARIABLE>: ${<VARIABLE>}
      # ${<VARIABLE>} = lue depuis le fichier .env automatiqu
      ement

    volumes:
      - ./<dossier_local>:/app
      # Monte ton code local dans le conteneur → modificali
```

ons visibles en temps réel (hot reload)

— Service avec image toute faite (Docker Hub) —

```
<nom_service_db>:
  image: <nom_image>:<version>
  # Exemples :
  # postgres:18      → PostgreSQL
  # mysql:9          → MySQL
  # mongo:8          → MongoDB
  # redis:8          → Redis
  # mariadb:11       → MariaDB

  environment:
    <VAR_REQUISE_PAR_IMAGE>: ${<VARIABLE_ENV>}
    # Chaque image a ses propres variables obligatoires
    # Consulter https://hub.docker.com pour les connaître

  ports:
    - "<port_machine>:<port_conteneur>"

  volumes:
    - <nom_volume>:<chemin_data_dans_conteneur>
    # Le chemin dans le conteneur est imposé par l'image
    # ex PostgreSQL : /var/lib/postgresql/data
    # ex MySQL       : /var/lib/mysql
    # ex MongoDB     : /data/db
    # ex Redis       : /data
```

— Volumes persistants —

```
volumes:
  <nom_volume>:
    # Déclare le volume ici pour que Docker le crée et le gère
    # Sans cette déclaration → erreur au démarrage
```

Commandes Docker du quotidien

Démarrage & arrêt

Action	Commande
Premier démarrage ou après modif d'un Dockerfile	<code>docker compose up --build</code>
Démarrage normal	<code>docker compose up</code>
Démarrage en arrière-plan (silencieux)	<code>docker compose up -d</code>
Arrêt (données conservées)	<code>docker compose down</code>
Arrêt + suppression des données	<code>docker compose down -v</code>
Redémarrer un service	<code>docker compose restart <service></code>

Logs & debug

Action	Commande
Logs de tous les services	<code>docker compose logs -f</code>
Logs d'un seul service	<code>docker compose logs -f <service></code>
Voir les conteneurs actifs	<code>docker compose ps</code>
Ouvrir un terminal dans un conteneur	<code>docker compose exec <service> sh</code>
Voir toutes les images	<code>docker images</code>
Voir tous les volumes	<code>docker volume ls</code>

Nettoyage

Action	Commande
Supprimer les images inutilisées	<code>docker image prune</code>
Tout nettoyer (images, conteneurs, cache)	<code>docker system prune -a</code>
Supprimer un volume spécifique	<code>docker volume rm <nom_volume></code>

Erreurs fréquentes & solutions

 Variables `.env` non trouvées


```
WARN: The "MA_VARIABLE" variable is not set. Defaulting to a blank string.
```

Cause : le `.env` n'est pas au bon endroit ou contient des espaces autour du `=`.

Solution :

```
ls -a # vérifier que .env est visible (les fichiers "." s
ont cachés sous Linux)
# Vérifier la syntaxe : MA_VARIABLE=valeur (sans espaces au
tour du =)
```

✗ `package.json not found` (ou équivalent)

```
npm error path /app/package.json
npm error enoent Could not read package.json
```

Cause : le projet n'est pas encore initialisé dans le dossier.

Solution : initialiser le projet d'abord (`npm init`, `pip init` ...), puis relancer `docker compose up --build`.

✗ Erreur de credentials Docker sous WSL2

```
error getting credentials - err: fork/exec /usr/bin/docker-
credential-desktop.exe
```

Cause : Docker Desktop essaie de gérer les credentials via Windows depuis WSL.

Solution :

```
echo '{}' > ~/.docker/config.json
```

✗ Service inaccessible depuis le navigateur

Cause : l'app écoute sur `localhost` à l'intérieur du conteneur, pas sur toutes les interfaces.

Solution : dans le `CMD` du Dockerfile, ajouter l'option `--host 0.0.0.0` (ou équivalent selon le framework).

✗ Modifications de code non prises en compte

Cause : les volumes ne sont pas montés correctement.

Solution : vérifier dans `compose.yaml` :

```
volumes:
  - ./<dossier_local>:/app ← doit pointer vers le bon dossier
```

✗ `must be a mapping` ou erreur YAML

Cause : problème d'indentation dans `compose.yaml`. YAML utilise des **espaces uniquement**, jamais de tabulations.

Solution :

```
cat -A compose.yaml # affiche les caractères cachés
# ^I = tabulation (interdit en YAML)
# ^M = retour chariot Windows (problème CRLF)
```

✗ Port déjà utilisé

```
Error: bind: address already in use
```

Cause : un autre processus utilise déjà ce port sur ta machine.

Solution : changer le port **gauche** dans le compose (le port de ta machine) :

```
ports:
  - "8081:8080" # ton app sera sur localhost:8081 au lieu de localhost:8080
```



Références officielles

Ressource	Lien
Docker — Documentation	https://docs.docker.com
Docker Compose — Référence	https://docs.docker.com/compose
Docker Hub — Images officielles	https://hub.docker.com
Dockerfile — Référence complète	https://docs.docker.com/reference/dockerfile
Bonnes pratiques Dockerfile	https://docs.docker.com/build/building/best-practices